To find the solution let us look

$$(A_1 \underbrace{(A_2 A_3)}_{48}) A_4$$

$$\underbrace{88}$$

$$158$$

as in question 3.a).

2

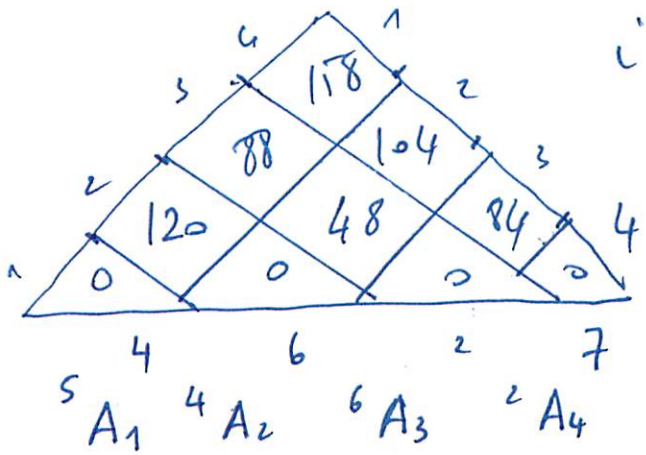
As predicted by Brute force.

7

8/8

ii)

Initialisation.



(2)

$$m[1,2] = \min_{1 \le k < 2} \left(m[1,k] + m[k+1,2] + p_0 p_k p_2 \right)$$

$$= m[1,1] + m[2,2] + p_0 p_1 p_2 = 120$$

$$m[2,3] = m[2,2] + m[3,3] + p_1 p_2 p_3 = 48$$

$$m[3,4] = p_2 p_3 p_4 = 84$$

$$m[1,3] = \min \begin{cases} m[1,1] + m[2,3] + p_0 p_1 p_3 \\ m[1,2] + m[3,3] + p_0 p_2 p_3 \end{cases} = 88$$ (3)

$$m[2,4] = \min \begin{cases} m[2,2] + m[3,4] + p_1 p_2 p_4 \\ m[2,3] + m[4,4] + p_1 p_3 p_4 \end{cases} = 104$$

$$m[1,4] = \min \begin{cases} m[1,1] + m[2,4] + p_0 p_1 p_4 \\ m[1,2] + m[3,4] + p_0 p_2 p_4 \\ m[1,3] + m[4,4] + p_0 p_3 p_4 \end{cases} = 118$$

(3)

c)

$$A_{i..j} = A_i \times \dots \times A_j \in \mathbb{R}^{p_{j-1} \times p_j}$$

i)

$m[i,j]$: optimal number of operations needed to compute $A_{i..j}$

$$A_{i..j} = A_{i..k} \; A_{k+1..j} \qquad i \le k < j$$ (2)

Our problem subdivides into two subproblems.

- Where to subdivide $A_{i..j}$? where is k? (2)
- How to parenthesise $A_{i..k}$ & $A_{k+1..j}$?

not needed!

It is clear that if we chose k in the parenthisation of $A_{i..j}$ then this ~~might~~ must be optimal for the parenthisation of $A_{1..n}$.

we can represent solution by binary trees and consider the optimality of subtrees. (6)

(2) $$m[i,j] = \begin{cases} 0 & i = j \\ \min_{i \le k < j} \left(m[i,k] + m[k+1,j] + p_{i-1} p_k p_j \right) & i < j \end{cases}$$

6/8

/25

(3)

a) $A_1^{5} \; A_2^{4} \; A_3^{6} \; A_4^{2}$ 4

$(A_1 A_2)(A_3 A_4)$: $5\times4\times6 + 6\times2\times7 + 5\times6\times7 =$
$120 + 84 + 210 = 414$

$A_1((A_2 A_3) A_4)$: $4\times6\times2 + 4\times2\times7 + 5\times4\times7 =$
$48 + 56 + 140 = 244$

$(A_1 (A_2 A_3)) A_4$: $4\times6\times2 + 5\times4\times2 + 5\times2\times7 =$
$48 + 40 + 70 = 158$

$((A_1 A_2) A_3) A_4$: $5\times4\times6 + 5\times6\times2 + 5\times2\times7 =$
$120 + 60 + 70 = 250$

$A_1(A_2 (A_3 A_4))$: $6\times2\times7 + 4\times6\times7 + 5\times4\times7 =$
$84 + 168 + 140 = 392.$

b)

i) $(A_1 \dots A_k)(A_{k+1} \dots A_n)$

start by doing the parenthisation inside $A_1 \dots A_k$.

and $(A_{k+1} \dots A_n)$ respectively. (2)

Hence $P(n) = \sum_{k=1}^{n-1} P(k)\,P(n-k)$. (2) 4

ii) $P(n) \geq P(n-1)\,P(1) + P(n-2)\,P(2)$

$P(1) = P(2) = 1$

$P(n) \geq P(n-1) + P(n-2)$. (2) 4

So $P(n)$ is larger than a Fibonacci sequence which grows exponentially. $\geq 2^{n/2}$. (2) 5/8

(2) (b) ii)

Compute $b_{1,j}$ $j = 1, \dots n$. (1)

to compute $b_{i,j} = b_{1,j} - b_{1,i-1}$ (2). } 2

Complexity of (1): $1 + \dots + n-1 = \frac{n(n-1)}{2}$

Complexity (1) $\# \{(j,i) \; ; \; i<j\} = \frac{n(n-1)}{2}$ } 4

Total: $O(n^2)$.

6

11

(2) $b_{ij} = \sum_{k=i}^{j} a_k \quad 1 \le i < j \le n.$

3

a) fn $a_k = k$.

$$b_{ij} = \underbrace{\sum_{k=i}^{j} k}_{1} = \sum_{k=1}^{j} k - \underbrace{\sum_{k=1}^{i-1} k}_{2} = \frac{j(j+1)}{2} - \frac{i(i-1)}{2}.$$

b)

i) Complexity of proposed algorithm ij.

$$\underbrace{\sum_{i=1}^{n}}_{\text{first fn loop}} \underbrace{\sum_{j=i+1}^{n}}_{\text{2nd fn loop}} \underbrace{(j-i)}_{\#\text{ additions in } a_i + a_{i+1} + \dots + a_j} = \sum_{i=1}^{n} \sum_{j=i+1}^{n} j - \sum_{i=1}^{n} \sum_{j=i+1}^{n} i$$

(2) By a) $\rightarrow$ $\begin{cases} j \to n \\ i \to i+1 \end{cases}$

$$= \sum_{i=1}^{n} \frac{n(n+1)}{2} - \frac{i(i+1)}{2}$$

6

$$- \sum_{i=1}^{n} i(n-i).$$

(3)

$$= \frac{n^2(n+1)}{2} - \sum_{i=1}^{n} i^2/2 - \sum_{i=1}^{n} i/2$$

$$- \sum_{i=1}^{n} in + \sum_{i=1}^{n} i^2$$

$$= \frac{n^3}{2} + n^2/2 - \frac{(2n-1)n(n+1)}{12} - \frac{n(n+1)}{4}$$

$$- \frac{n^2(n+1)}{2} + \frac{(2n-1)n(n+1)}{6}.$$

$$= \underbrace{\left(\frac{1}{2} - \frac{1}{6} - \frac{1}{2} + \frac{1}{3}\right)}_{1/6} n^3 + O(n^2) = O(n^3).$$

(1) 3/8

iii)

$T(n)$: # multiplications

$T(1) = 1$

$T(n) = 4\,T(n/2) + \underbrace{O(n)}_{\text{add \& shift}}$ | (4)

By Master theorem $\Rightarrow T(n) = O(n^2)$.

It is of the same order as the standard multiplication which takes n^2 multiplications and n^2 additions and n shifts. $\Rightarrow O(n^2)$ complexity. | (4) | 4

c)

$ab = 2^n a'b' + 2^{n/2}\left((a'+a'')(b'+b'') - a'b' - a''b''\right) + a''b''$

i) Multiply1(a,b)

differs from Multiply(a,b) in that we only do need

Multiply1(a', b')

Multiply1$((a'+a''), (b'+b''))$

Multiply1(a'', b''). | 4

ii) $T_1(n) = 3\,T_1(n/2) + O(n)$ (2)

$\Rightarrow T_1(n) = O(n^{\log 3}) = O(n^{1.585})$ (2) | 4

which quicker than before. | 2/8

① /20

a) $a = (a_1 \dots a_n)$
$b = (b_1 \dots b_m)$

to compute ab we have to perform all products $a_i \times a_j$ $i = 1 \dots n$, $j = 1 \dots m$.

$\Rightarrow$ n^2 multiplications. 2

b) i) $(a \times b) = (a' 2^{n/2} + a'') \cdot (b' 2^{n/2} + b'')$

$= a'b' 2^n + 2^{n/2}(a'b'' + a''b') + a''b''$ 2

ii) Multiply (a, b).

if $|a| = |b| = 1$ then $a.b$. ← $a_1 \cdot b_1$

else.

$a' = (a_1 \dots a_{n/2})$

$b' = (b_1 \dots b_{n/2})$

$b'' = (b_{n/2+1} \dots b_n)$

$a'' = (a_{n/2+1} \dots a_n)$.

2 | Shift(Multiply(a', b'), n) +
shift(Multiply(a', b'') + Multiply(a'', b'), $n/2$) +
Multiply (a'', b'') .

let Size(a) = number of digits in a.

one shift(a, k) the function that shift the digits of a k-steps to the left.

shift$(a_1 \dots a_n) = (a_1 \dots a_n \underbrace{0 \dots 0}_{k \text{ times}})$.

+1 Bonus if complete procedure

4

1/8